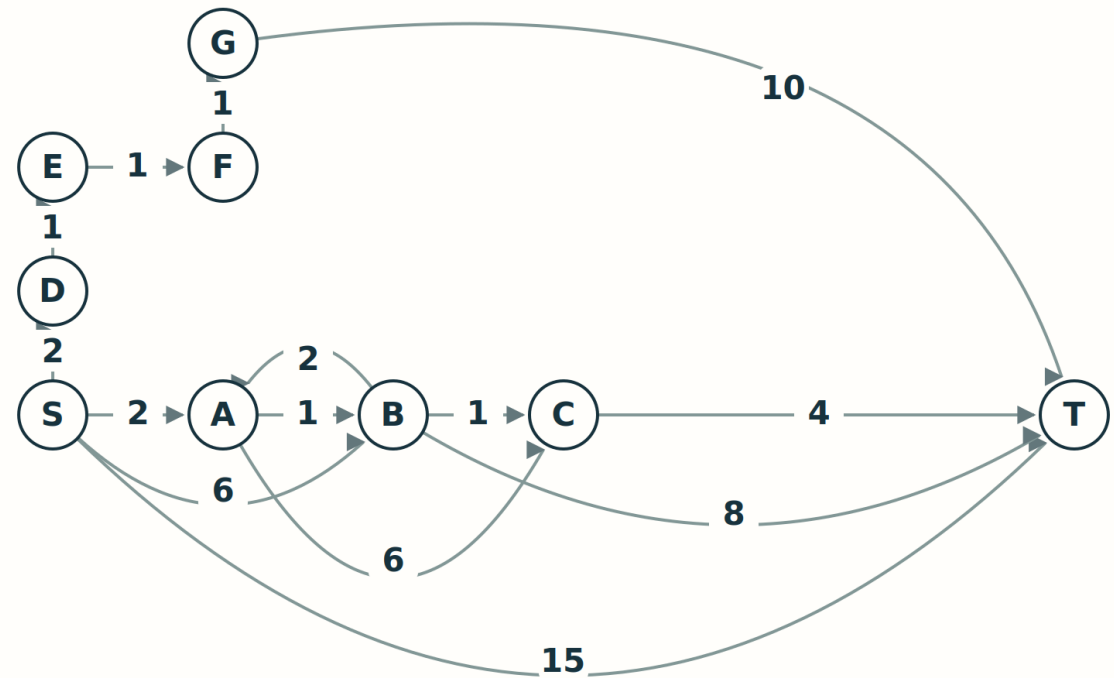


Dijkstra and A*

When can a route
be finalized?

Conjecture → counterexample → repair



Find a route. Then justify stopping.

Finding a route is only half the problem.

Navigation

A current position is not a route.
What path minimizes the chosen cost?

Networks

Which next hop and outgoing interface?
OSPF builds shortest-path trees.

Agents

Which sequence of valid moves?
The state must encode what matters.

Can you certify that no cheaper route exists?

First, agree on the problem.

minimize $C(P)$ = sum of edge costs on P

Given

A finite directed graph $G=(V,E)$.
Start s , target t .
Fixed, nonnegative edge costs.

Return

The minimum cost.
A route attaining it.
Or: all distances from s .

What should happen if t is unreachable? What if $s = t$?

A graph model is not a storage format.

Model

Intersections, grid cells, or regions?
Choose states and valid transitions.

Store

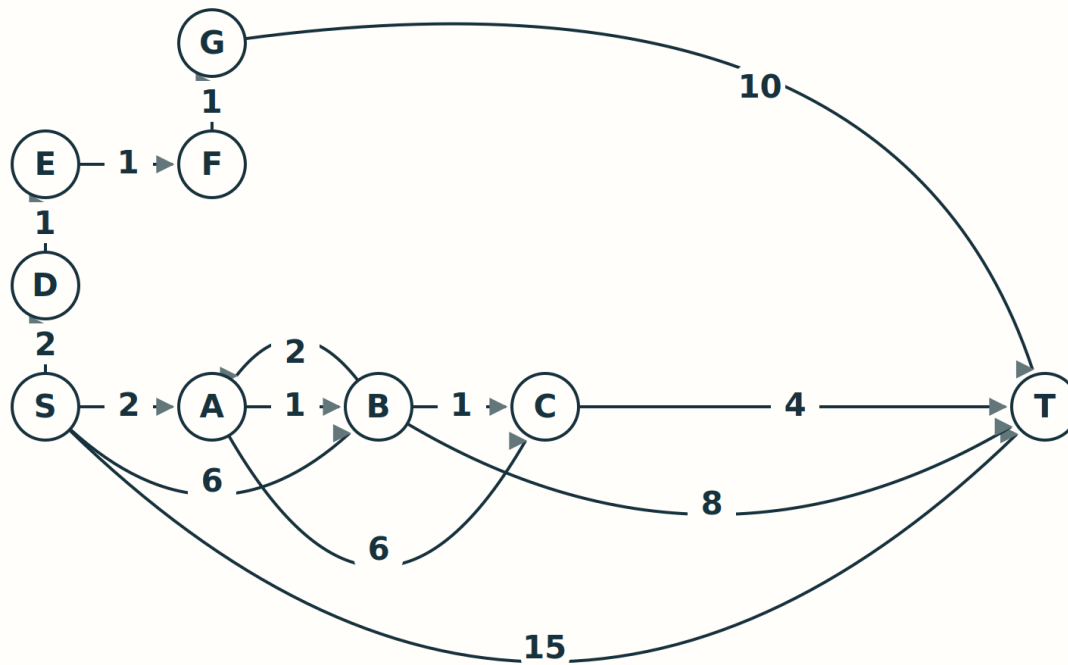
Adjacency list:
 $O(n+m)$.
Matrix: $O(n^2)$ entries.

Generate

For an implicit grid,
compute
neighbors when
needed.

For the live code: a dictionary of outgoing (neighbor, cost) pairs.

Could we simply try every route?



First conjecture

Enumerate candidate routes.
Add their edge costs.
Keep the cheapest.

What does the cycle $A \rightarrow B \rightarrow A$ do to this plan?

Simple does not mean few.

```
for v, weight in graph[u]:  
    if v not in on_path:  
        explore(v, cost + weight)
```

```
# on_path belongs to this route.  
# It is not a global visited set.
```

Predict before running

How many complete routes?

When would this approach become infeasible?

Many different routes reach the same intermediate state.

Which prefixes are already dominated?

Same state B: cost 6 versus cost 3

Discard

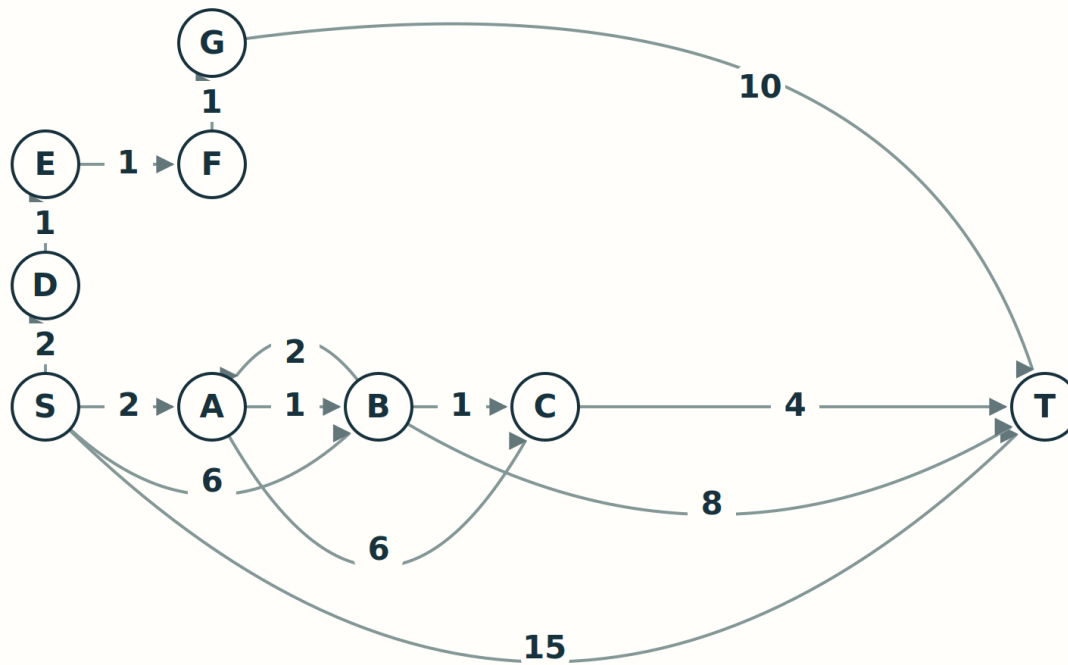
The more expensive prefix has no advantage for the remaining state-based journey.

Remember

$g(v)$: best discovered cost.
 $\text{parent}(v)$: predecessor for that improving route.

Best discovered is not yet proved optimal.

What should we process next?



After expanding S

A: 2
B: 6
D: 2
T: 15

Choose the smallest g.

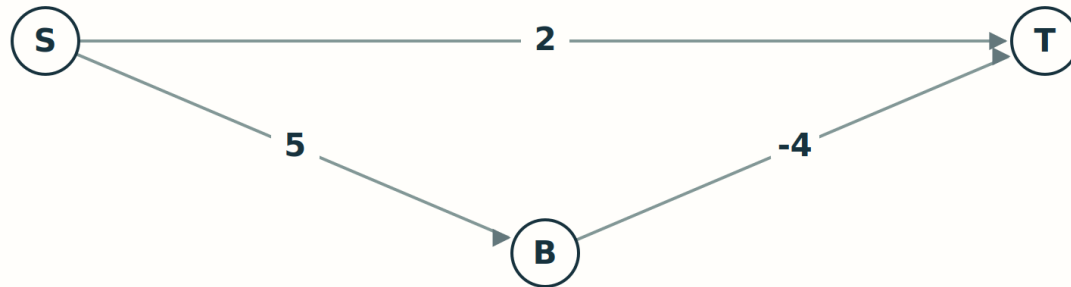
candidate = $g(u) + c(u,v)$ · accept only a strict improvement

Why is the smallest label final?

- 1 Assume** There is a cheaper route to the selected u .
- 2 Find** Take its first unfinalized node y and predecessor x .
- 3 Compare** Relaxation exposed y ; a nonnegative suffix cannot lower cost.

Minimum extraction forces $g(u) \leq g(y) \leq \text{optimal cost to } u$.

Attack the assumption, not the conclusion.



Predict the failure

Which goal cost is extracted first?

Can a later suffix make a route cheaper?

Which exact line of the finalization argument breaks?

Make the proof executable.

```
candidate = popped_g + weight
if candidate < dist[v]:
    dist[v] = candidate
    parent[v] = u
    heappush(queue,
              (candidate, ticket, v))
```

Three decisions

Accumulate route cost.
Keep only strict improvements.
Schedule the improved route.

A queue entry is a proposal about a route, not a certificate.

What happens to the old $B = 6$ entry?

```
popped_g, _, u = heappop(queue)
if popped_g != dist[u]:
    continue
```

```
# B=3 superseded B=6.
# C=4 superseded C=8.
```

Two separate records

Keep the new route.
Leave the old record in the heap.
Discard it when it reaches the top.

Update parent(v) on every improvement, not just first discovery.

Stop at discovery—or at certification?

Discovery

A route to T exists.
Its cost is only an upper bound.

Extraction

After the stale guard,
minimum-g T is certified.

Continue

With no target stop,
finalize every reachable distance.

Do not label every tentative distance “shortest” after an early stop.

Can each frontier node have one entry?

```
if v in queue:
    queue.decrease_key(v, candidate)
else:
    queue.insert(v, candidate)

# heap array + node→position map
```

What changes?

The search rule does not.
The queue bookkeeping does.
Every swap repairs the map.

Use the prepared indexed heap; inspect one decrease-key operation.

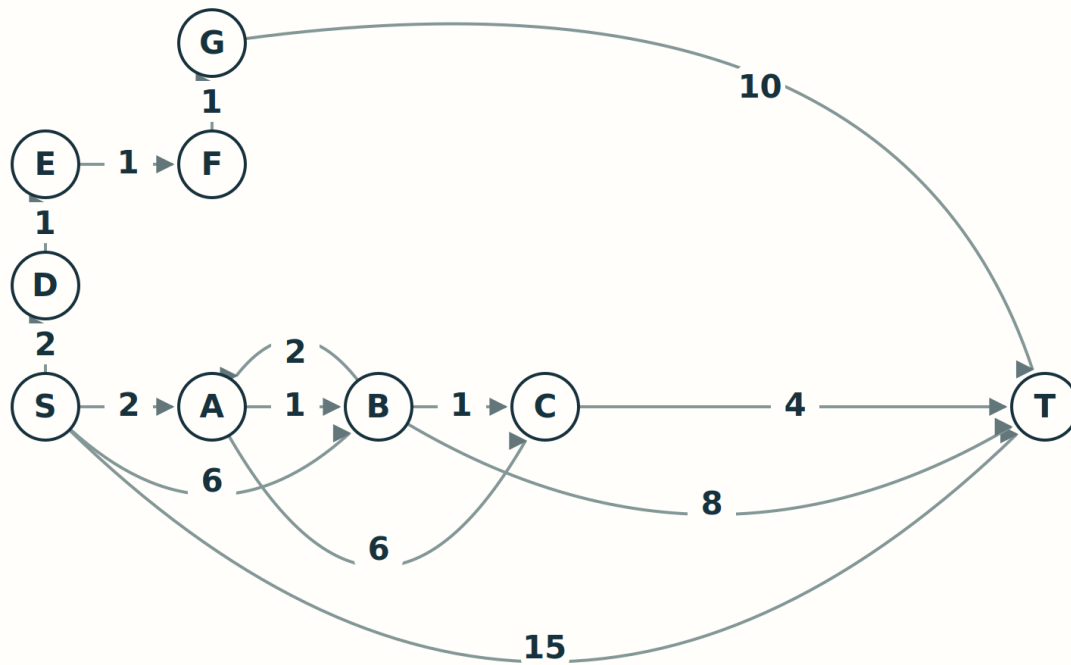
Smaller queues do not prove faster runs.

Variant	Time	Extra memory
Lazy / binary heap	$O(n + m \log(m+1))$	$O(n+m)$
Eager / indexed heap	$O((n+m) \log(n+1))$	$O(n)$

Same search rule. Different update costs, queue size and implementation constants.

$n = |V|$, $m = |E|$. For simple graphs, $\log m = O(\log n)$.

Which term in the priority mentions T?

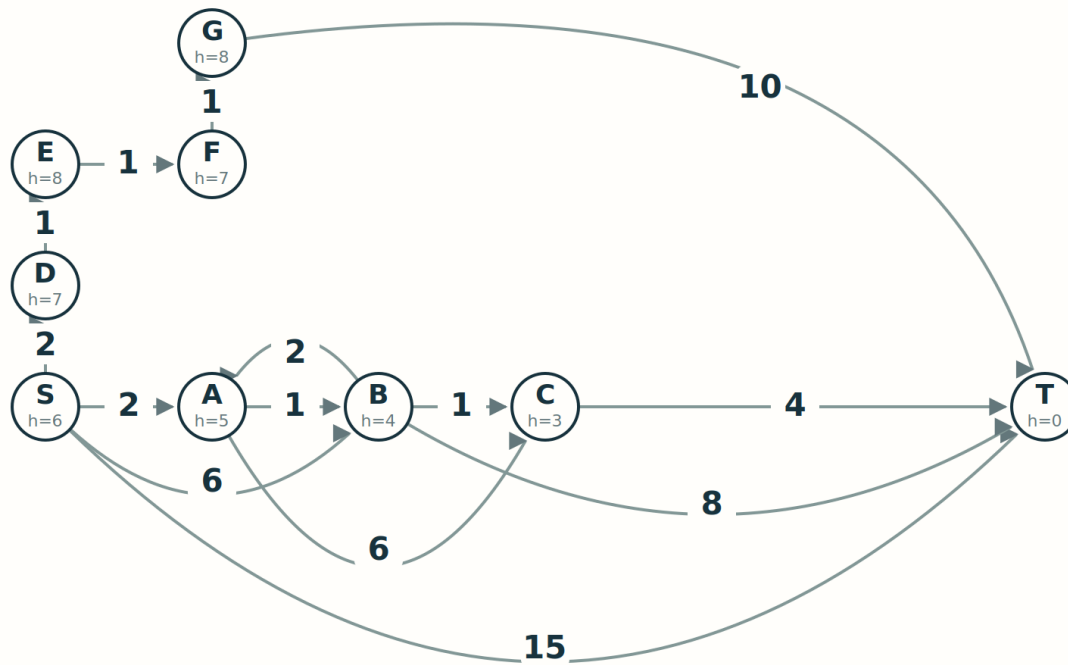


Priority = $g(v)$

The detour is cheap so far.
Dijkstra processes it before certifying $T=8$.

Dijkstra respects edge direction. It does not use target information to prioritize.

What if we use only the estimated distance?



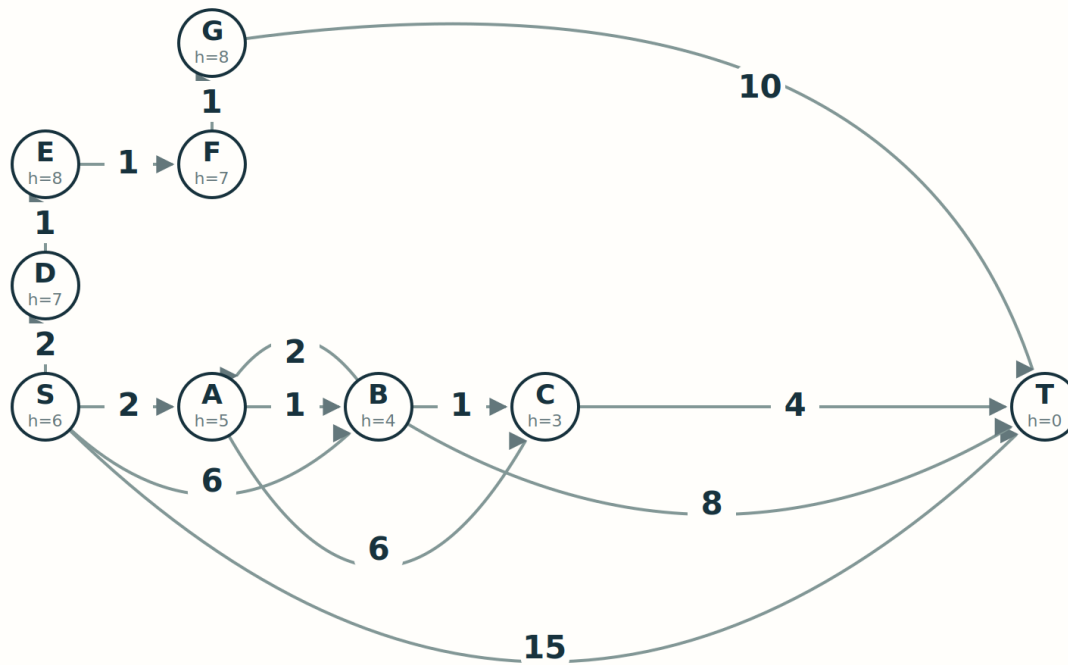
Priority = $h(v)$

The goal has $h(T)=0$.
Its first discovered route
costs 15.

What did we forget?

Estimated remaining cost cannot replace accumulated route cost.

Keep the past. Estimate the future.



Selected (g, f)

S: (0, 6)

A: (2, 7)

B: (3, 7)

C: (4, 7)

T: (8, 8)

D remains at $f = 2 + 7 = 9$. Setting $h = 0$ recovers Dijkstra.

What promise must the estimate make?

$$0 \leq h(v) \leq h^*(v) \quad \text{and} \quad h(T) = 0$$

Break it first

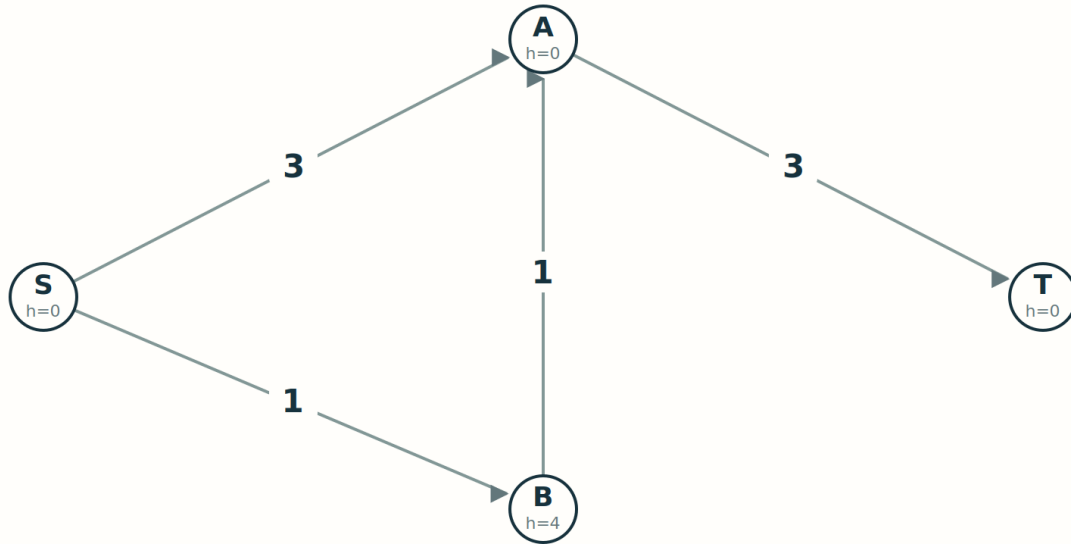
Change $h(A)$ to 20.
Can an inflated estimate
hide the optimal route?

Admissibility

Never overestimate
the true remaining
optimal cost.

The proof needs an appropriate optimal-prefix representative on the frontier.

Does admissibility let us close forever?



Predict, then trace

A is expanded at $g=3$.
B later proposes A at $g=2$.

Should we ignore that improvement?

All h values are admissible. Inspect the edge $B \rightarrow A$.

Recover the finalization argument.

$$h(u) \leq c(u,v) + h(v)$$

Local condition

The estimate cannot drop by more than the cost of taking an edge.

Reweighted cost

$c_h(u,v) = c(u,v) + h(v) - h(u)$
Consistency means $c_h \geq 0$.

Along a route: $g_h(v) = f(v) - h(S)$. The ordering is Dijkstra again.

A small code change. A new proof obligation.

```
priority = candidate + h(v)
heappush(queue,
            (priority, ticket, candidate, v))

f, _, queued_g, u = heappop(queue)
if queued_g != dist[u]:
    continue
```

Keep the distinctions

f chooses what comes next.
g records the route cost.
A better g may reopen a state.

Do not compare f against g in the stale-entry guard.

Which improvements did we actually obtain?

Main graph	Cost	Expansions	Stale skips	Peak queue
Dijkstra / lazy	8	8	2	6
Dijkstra / eager	8	8	0	4
A*	8	4	0	6

Fewer expansions, smaller queues, and lower runtime are different claims.

Goal extraction is not an expansion. These counts describe this fixture, not every graph.

Compare before concluding.

Scaling

Complete graphs:
when does
enumeration
exhaust the budget?

Bookkeeping

Lazy versus eager:
did fewer queue
records
produce a faster run?

Heuristics

Same grid, different h:
what changes when
walls
or movement rules
change?

Use the worked notebook and saved results. Measure search separately from rendering.

What justifies an irreversible decision?

- 01** Why is discovering the goal not enough?
- 02** When may an expanded state be closed permanently?
- 03** Why can we skip stale entries and still reopen states?

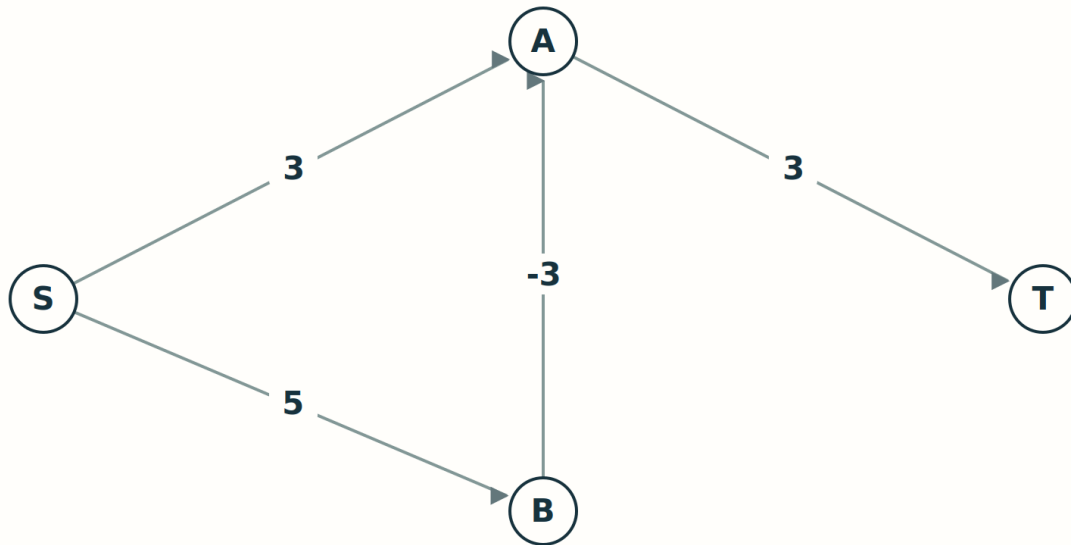
Enumerate → discard dominated prefixes → prove finalization → inform → test.

Appendix: the complete finalization proof

- 1 Induction** Every node in S has $\text{dist}(x) = \text{delta}(x)$. Choose minimum current u outside S .
- 2 Crossing** On an optimal route to u , let y be the first node outside S and x its predecessor. Relaxation from x gives $\text{dist}(y) = \text{delta}(y)$.
- 3 Contradiction** Nonnegative suffix: $\text{delta}(y) \leq \text{delta}(u)$. Minimum selection: $\text{dist}(u) \leq \text{dist}(y)$. Thus $\text{dist}(u) \leq \text{delta}(u)$, while every label is a real route cost.

The heap supplies the minimum candidate; the invariant supplies correctness.

Appendix: the same failure in another graph



$$c_h = c + h(v) - h(u)$$

$S \rightarrow B$ becomes 5.
 $B \rightarrow A$ becomes -3.

All fixed-endpoint routes
shift by one constant.

An inconsistent heuristic can create negative reweighted edges.

Appendix: count paths, not just vertices.

Vertices n	Complete simple s-to-t paths
5	16
7	326
9	13,700
11	986,410

For complete directed graphs: sum over $k=0,\dots,n-2$ of $(n-2)! / (n-2-k)! .$

Appendix: a heap is a design choice.

Binary

$O(\log n)$ inserts, decreases and extracts with an indexed heap.

d-ary

Lower height, but more children to compare on extraction.

Fibonacci

Amortized $O(1)$ decrease.
 $O(\log n)$ extraction.
Classical SSSP:
 $O(m+n \log n)$.

There is no universal graph-size threshold at which the most intricate heap wins.

Appendix: the movement model is part of h.

Four neighbors

Unit cardinal moves.
Without walls:
 $|dx| + |dy|$.

Eight neighbors

All moves cost 1.
Without walls:
 $\max(|dx|, |dy|)$.

Weighted moves

Use a proven lower bound
with the correct units
and movement costs.

A diagonally adjacent goal refutes Manhattan distance when a diagonal costs 1.

Appendix: beyond the classical bound

$$O(m \log^{2/3} n)$$

Duan, Mao, Mao, Shu and Yin (2025)
Directed SSSP · real nonnegative edge weights
Deterministic · comparison-addition model

A model-specific research result, not an implementation or runtime claim for this lecture.

References / algorithmic foundations

- R1** Dijkstra (1959). A Note on Two Problems in Connexion with Graphs.
ir.cwi.nl/pub/9256/9256D.pdf
- R2** Python documentation. heapq: priority queue implementation notes.
docs.python.org/3/library/heapq.html
- R3** UC Berkeley CS188. Informed Search.
inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html
- R4** Stanford CS221. Search II (2023), proofs and reweighting.
stanford-cs221.github.io/spring2023-extra/modules/search/search2.pdf

References / applications and perspective

- R5** NetworkX documentation. all_simple_paths and factorial growth.
networkx.org/documentation/stable/reference/algorithms/
- R6** Moy (1998). OSPF Version 2. RFC 2328, section 16.1.
rfc-editor.org/rfc/rfc2328.html#section-16.1
- R7** Chen et al. (2007). Priority Queues and Dijkstra's Algorithm.
www3.cs.stonybrook.edu/~rezaul/papers/TR-07-54.pdf
- R8** Duan et al. (2025). Breaking the Sorting Barrier for Directed SSSP.
arxiv.org/abs/2504.17033v2